SUPPLEMENTARY MATERIALS: ROBUST QUANTUM DOTS CHARGE AUTOTUNING USING NEURAL NETWORK UNCERTAINTY

A PREPRINT

Victor Yon [*1,2,3], **Bastien Galaup** [1,2,3], **Claude Rohrbacher** [2,3,4], **Joffrey Rivard** [2,3,4], **Clément Godfrin** [5], **Ruoyu Li** [5], **Stefan Kubicek** [5], **Kristiaan De Greve** [5], **Louis Gaudreau** [6], **Eva Dupont-Ferrier** [2,3,4], **Yann Beilliard** [1,2,3], **Roger G. Melko** [7,8], **and** **Dominique Drouin** [1,2,3]

[1]Institut Interdisciplinaire d'Innovation Technologique (3iT), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

[2]Laboratoire Nanotechnologies Nanosystèmes (LN2) — CNRS 3463, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

[3]Institut quantique (IQ), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

[4]Département de physique, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

[5]IMEC, Kapeldreef 75, 3001 Leuven, Belgium

[6]National Research Council Canada, Quantum and Nanotechnologies Research Center, ON, Ottawa, Canada, K1A 0R6

[7]Department of Physics and Astronomy, University of Waterloo, Waterloo, ON, Canada, N2L 3G1

[8]Perimeter Institute for Theoretical Physics, Waterloo, ON, Canada, N2L 2Y5

October 14, 2024

[*]victor.yon@usherbrooke.ca

S1 Datasets

Table S1: Summary of dataset specifications. SET and QPC stand for single-electron transistor and quantum point contact, respectively.

Dataset short name	**Si-SG-QD**	**GaAs-QD**	**Si-OG-QD**
Dataset full name	Silicon split gate quantum dot	Gallium arsenide quantum dot	Silicon overlapping gate quantum dot
Current measurement method	SET	QPC	SET
Pixel size	$1\,\mathrm{mV}$	$2.5\,\mathrm{mV}$	$2\,\mathrm{mV}$
Detection area offset	6 pixels	7 pixels	6 pixels
Prior line distance	$30\,\mathrm{mV}$	$16\,\mathrm{mV}$	$30\,\mathrm{mV}$
Prior line slope (0° = horizontal, 90° = vertical)	75°	45°	-10°
Use last-line validation	No (not necessary)	Yes	Yes
Number of diagrams with transition line annotations (used for line detection training and test)	17	9	12
Number of diagrams with transition line and charge area annotations (used for autotuning test)	9	9	9
Number of patches (18×18 pixels each)	72,182	4349	48,081
Patch class ratio (no-line / line)	33.6	3.3	8.2
References	[1]	[2]	[3]

S1.1 Data selection

The stability diagrams used in this article are the result of experimental measurements performed in prior studies on quantum dot (QD) devices [1, 2]. Thus, most of the measurements were not suitable for this work, and a selection was made. Only stability diagrams meeting the following criteria were included in the final datasets:

General criteria:

- All diagrams of the same dataset are measured using similar devices. Hardware differences are acceptable between datasets, but consistency is required for each independent training.
- A similar measurement step size is required between G1 and G2. A step size missmatch between the two measured gates makes pixel interpolation less reliable.
- Double-QD diagrams are excluded.

Transition line detection patch datasets:

- At least one transition line must be visible or partially visible in the diagram.

Autotuning datasets:

- At least three transition lines must be visible or partially visible in the diagram.
- A clearly identifiable empty regime must be in the measured range (no line for at least two times the average space between visible lines).

S1.2 Data processing

The long-term goal of this project is to implement an automated control loop as close as possible to the quantum device. Thus, we want to minimize the preprocessing of the input data to avoid any computation overhead inside the cryogenic environment. This is why we do not compute the derivative on the patches or perform complex data processing, such as the method proposed by Czischek et al. [4]. Furthermore, most statistical methods require a large sample size to be reliable, which is incompatible with our small patch measurement strategy.

To improve the convergence of the models, we normalized the input model to a range between 0 and 1 based on the smallest and largest current value of each patch.

S1.3 Diagram annotation

We manually annotated the transition lines and the charge regime areas using *Labelbox* [5] at the diagram level. When the transition lines were mixed with background noise, we used the derivative of the images to place more accurate annotations. We only annotated the lines that we could visually identify. We never filled the gap between two sections of a line, even when it was clear that they were a continuation of the same transition.

The voltage space was segmented into charge regime areas annotated from 0 to 3 electrons, then the voltage space corresponding to more than 3 electrons was annotated as "*4+*". The location of each area was deduced from the position of the transition lines and the knowledge that the empty regime was located at the bottom left of the images. When the regime of a section was ambiguous or difficult to identify (too noisy, fading line, few pixels around a line), no annotations were set, and the regime of this voltage space was considered as "*unknown*" during the experiment (examples in Figure 2e,f). When an autotuning ended in an "*unknown*" regime, it was always considered as a failure.

Annotating the transition lines and charge regimes of one stability diagram took between 5 and 30 min, depending on their complexity. It took approximately $10\,\mathrm{h}$ to annotate the 38 stability diagrams used in this study.

S1.4 Patch segmentation and automatic class labeling

The patch size was fixed at 18×18 pixels as a tradeoff between measurement time and classifier performance. With a smaller patch size, it becomes difficult to distinguish a transition line from noise. A larger patch would directly increase the time required to measure it, reducing the exploration efficiency.

To train and test the neural networks (NNs), we split the diagrams into patches by sliding an 18×18 window with 10 overlapping pixels between each step. We then automatically assigned a class to each patch using the manual transition line annotations set previously (see Section S1.3). If at least one line intersected with the detection area (Figure 3b), the patch was categorized as "*line*"; otherwise, it was categorized as "*no-line*".

S1.5 Subset splitting

Each patch dataset was split into three mutually exclusive subsets: training, validation, and test.

Each subset was used for a specific purpose:

Training set: used to train the line detection model.

Validation set: used after training to select the step corresponding to the best model (green stars in Figures S5) and to calibrate the confidence thresholds (Figures S6).

Test set: used to evaluate the line classification accuracy and the autotuning success rate after training and calibration.

In the case of training with cross-validation (Figure S4), the subsets were split as follows:

Training set: composed of $90\,\%$ of the patches, randomly selected among the training diagrams.

Validation set: composed of $10\,\%$ of the patches, randomly selected among the training diagrams.

Test set: composed of every patch of the testing diagram.

In the case of training without cross-validation, the subsets were split as follows:

Training set: composed of $70\,\%$ of the patches, randomly selected among every diagram.

Validation set: composed of $10\,\%$ of the patches, randomly selected among every diagram.

Test set: composed of $20\,\%$ of the patches, randomly selected among every diagram.

S1.6 Stability diagram samples

All of stability diagrams used in this study can be downloaded from Yon et al. [6]. To illustrate the transition line annotation methodology and demonstrate the diagram diversity of each dataset, two diagrams of each dataset are presented in Figure S1, Figure S2, and Figure S3.

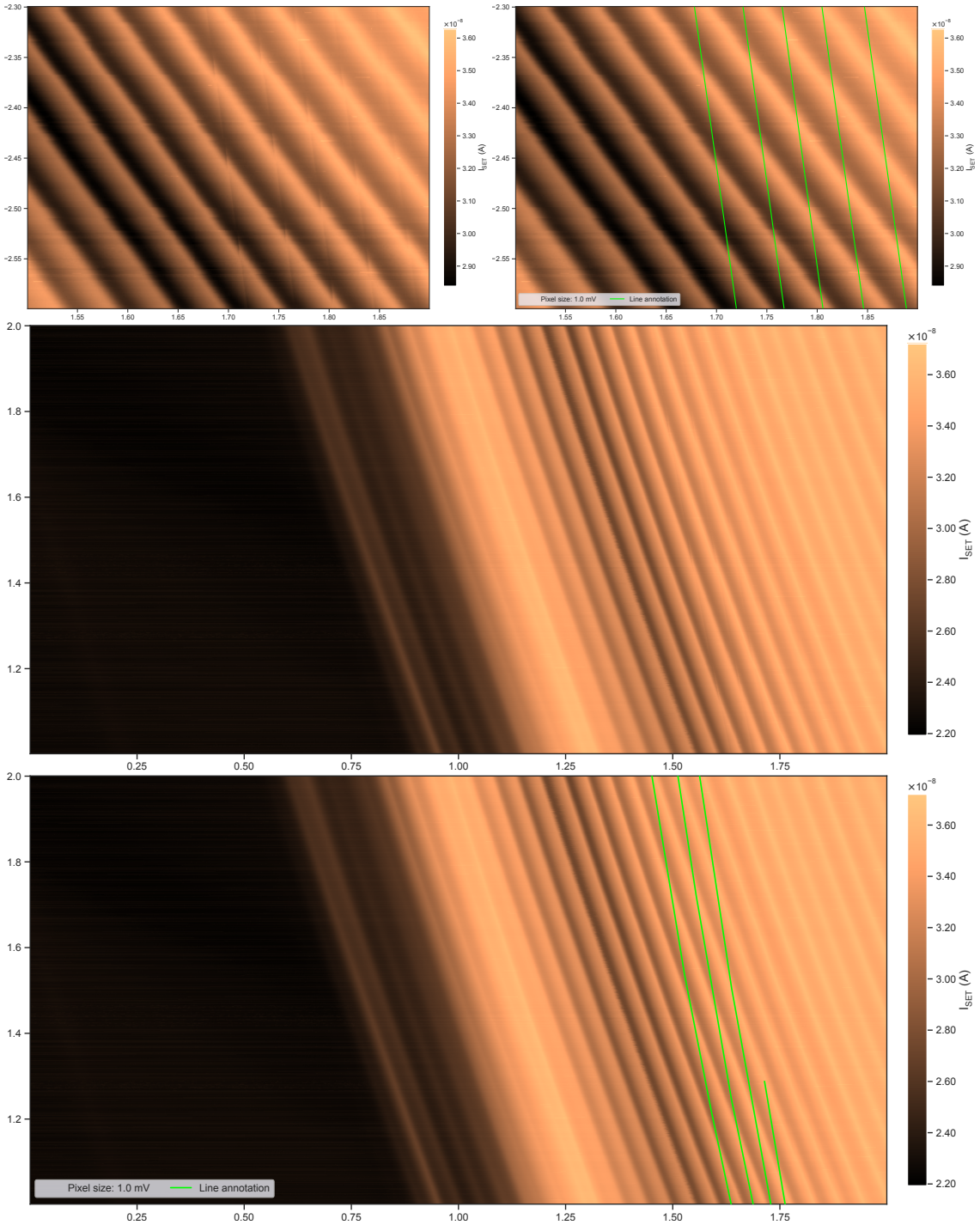


Figure S1: Two silicon split gate (Si-SG-QD) stability diagram examples, without and with transition line annotations. The x-axes correspond to the G1 gate voltage, and the y-axes correspond to the G2 gate voltage.

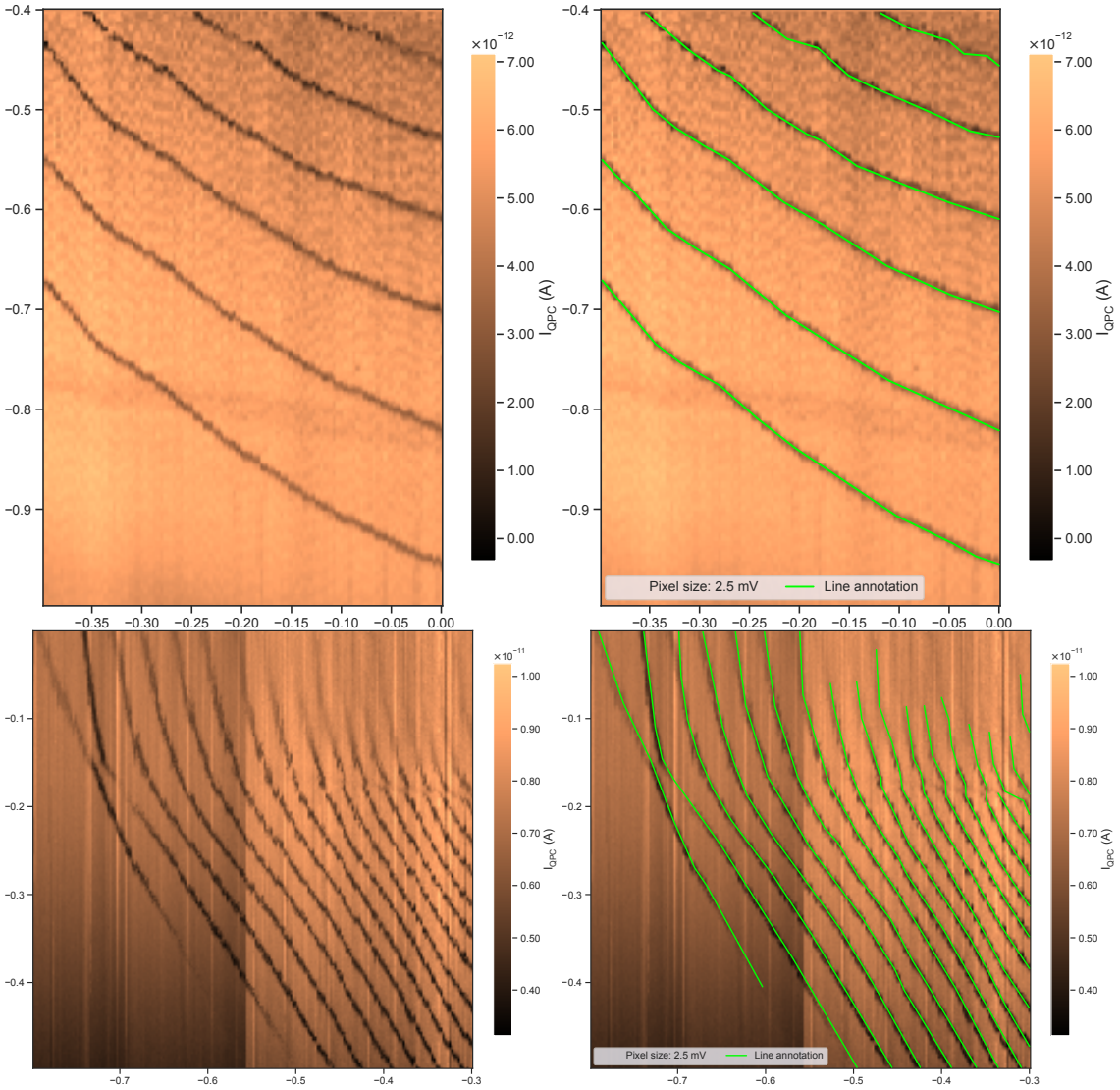


Figure S2: Two gallium arsenide (GaAs-QD) stability diagram examples, without and with transition line annotations. The x-axes correspond to the G1 gate voltage, and the y-axes correspond to the G2 gate voltage.

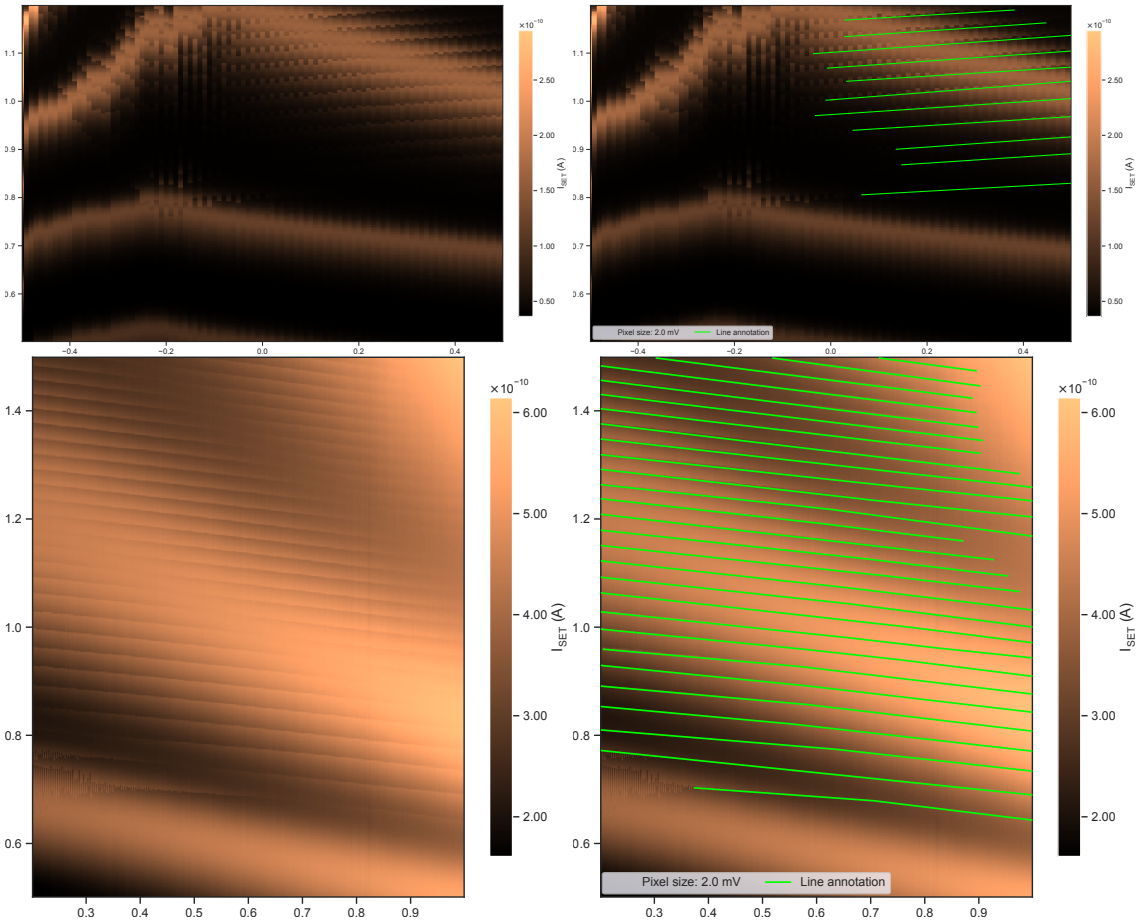


Figure S3: Two silicon overlapping gate (Si-OG-QD) stability diagram examples, without and with transition line annotations. The x-axes correspond to the G1 gate voltage, and the y-axes correspond to the G2 gate voltage.

S2 Line detection methodology

Table S2: Model architectures and training meta-parameters used in this study.

Meta-parameters	**FF**	**CNN**	**BCNN**
Number of training updates	15,000	30,000	
Layers description	Fully connected: - 400 - 100	Convolutions: - 4×4 kernel, 12 channels - 4×4 kernel, 24 channels Fully connected: - 200 - 100	
Number of free parameters	170,201	367,267	734,534
Loss function	Binary cross-entropy		Binary cross-entropy + complexity cost [7]
Optimizer	Adam [8]		
Learning rate	0.0005	0.001	
Dropout rate	60 %		0 %
Batch size	512		
Number of inferences per patch (N in Equation 2)	1		10

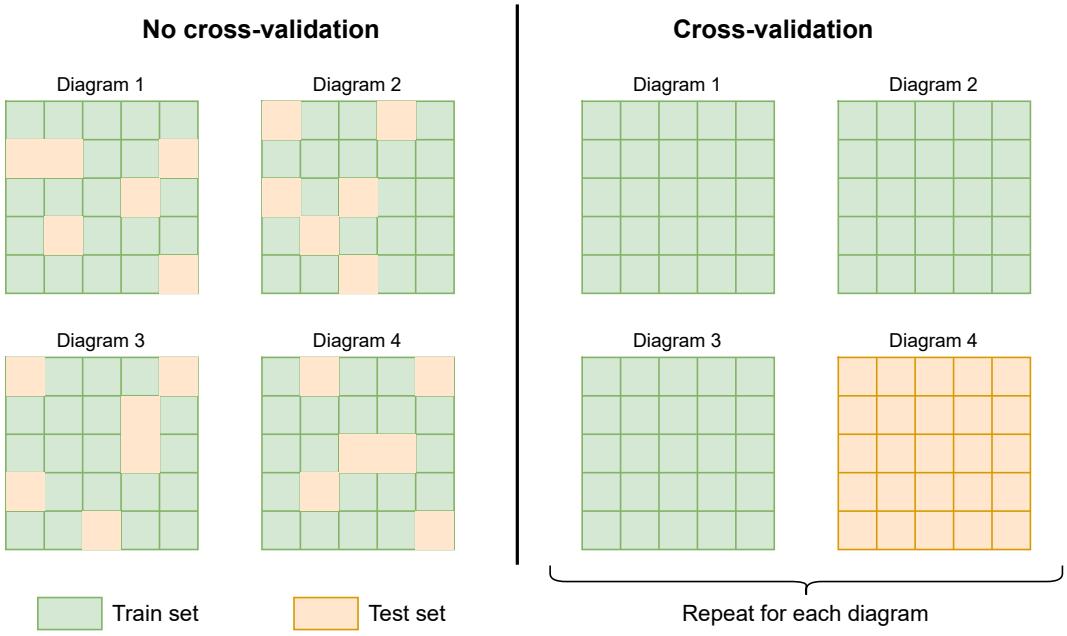


Figure S4: Diagram cross-validation methodology schematic. The left panel represents the "No cross-validation" method, where every diagram is split into patches, then a random set of patches is used for training, and the rest are used for testing. The right panel represents the "Cross-validation" method [9], where every diagram is split into patches, then the patches of one diagram are used for testing, while the patches of the others are used for training. This process is repeated for every diagram, and the performance is averaged.

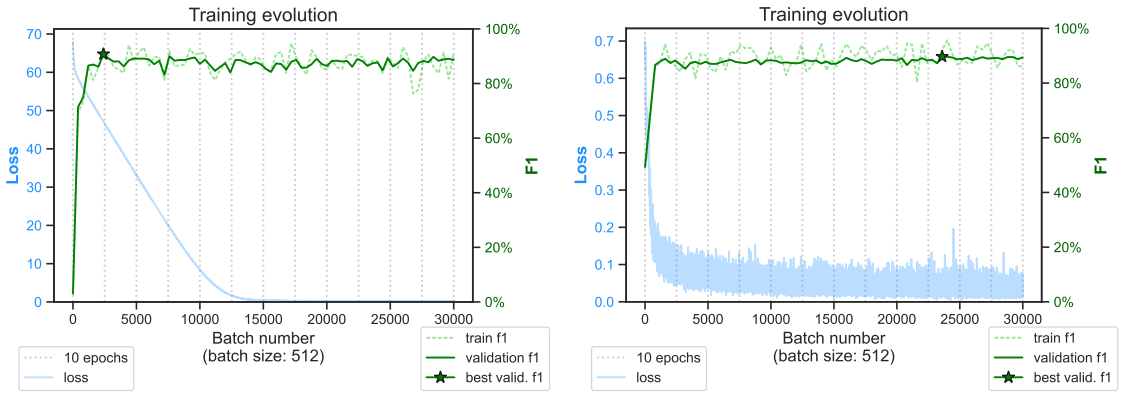
S2.1 Model training

Each NN was trained using PyTorch [10] v2.1 in a supervised manner. We also used the *BLiTZ* library [11] to create the Bayesian layers, sample the parameters, and compute the parameter updates using *Bayes-by-Backprop* [7] for the Bayesian convolutional neural networks (BCNNs). The training parameters and the NN architectures are described in Table S2. The source code used to obtain the results presented in this article is available on GitHub [1], and the output files (including the trained model parameters) can be downloaded from Yon et al. [12].

The imbalanced class ratio ("*line*" / "*no-line*") caused convergence issues, since the loss could be easily reduced by predicting only the most frequent class ("*no-line*"). We worked around this problem by re-balancing the patch dataset using weighted sampling for each batch.

The meta-parameters presented in Table S2 were selected based on a grid search and informed guesses. It is likely that these meta-parameters could be optimized and fine-tuned to improve the accuracy, accelerate the training, and reduce the number of free parameters. However, the scope of this study is to demonstrate the feasibility of the autotuning procedure and the benefits of using model uncertainty, not to achieve the best possible performance.

[1] Git repository: github.com/3it-inpaqt/dot-calibration-v2/tree/offline-article



(a) Bayesian convolutional neural network (BCNN)

(b) Convolutional neural network (CNN)

(c) Feed-forward neural network (FF)

Figure S5: Examples of training progress for the silicon split gate (Si-SG-QD) dataset using the different models and the cross-validation method. The F1 score is the harmonic mean of the precision and recall.

S2.2 Model uncertainty and confidence threshold calibration

To estimate and use NN uncertainty, we made three important decisions: what model to use, what confidence metric to compute, and how to define the confidence threshold under which the inference should not be considered reliable.

We evaluated a standard convolutional neural network (CNN) and feed-forward neural network (FF) as references, since these are the most simple and well-established NNs for image pattern recognition. Such models are not specifically designed or trained to estimate uncertainty. Several studies [13–16] have suggested that classical NNs are generally overconfident and provide badly calibrated confidence scores. For the sake of simplicity, we defined the confidence score as the distance between the output and the closest class (Formula 1). This score is normalized to a range between 0 and 1 to be easily interpretable and comparable as a confidence percentage.

We chose to implement a BCNN using variational inference [17, 18] and trained it using the Bayes-by-Backprop [7] method. This choice was motivated by previously promising results [19, 20], and the library [11] was compatible with PyTorch [10]. The confidence score was computed as a normalized standard deviation (Formula 2) from 10 inferences of the same input with newly sampled parameters.

A model is well calibrated if the confidence score accurately expresses the probability of making an error; for example, 20 % of the classifications with 80 % confidence should be wrong. However, a trained NN is never perfectly calibrated, and the calibration task is challenging for several reasons. First, NN training involves some stochastic processes (parameter initialization, stochastic gradient descent, etc.) that lead to different results. Therefore, two identical models trained separately could have very different and unpredictable miscalibration issues. Secondly, within one model, each predicted class could necessitate independent calibration (Figures S6). We also suspect that the unbalanced class distribution of the dataset could amplify the problem. Finally, calibrating the confidence score using the training set induces a bias, since the out-of-distribution examples are (by definition) not represented. Correctly expressing the distributional uncertainty in the model score is critical, especially for the cross-validation testing method.

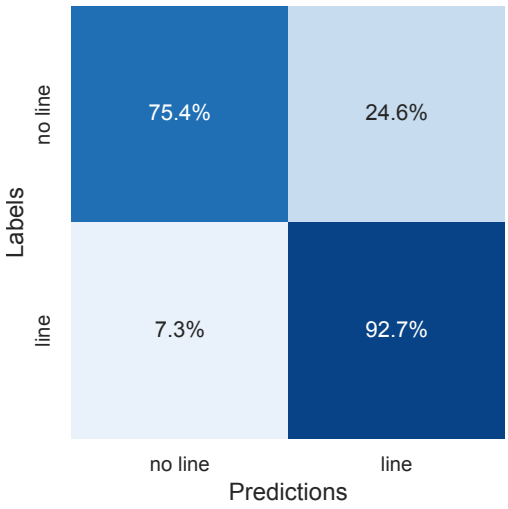
(a) Evolution of the threshold score (Formula 3) when the threshold is varied. The threshold with the lowest score value is selected as the optimal threshold for each class.

(b) Histogram of the model classification error as a function of the confidence score. The confidence thresholds obtained from (a) are represented by the vertical dotted lines. The top panel shows the distribution for both classes combined, while the bottom panels show the distribution for each class independently.

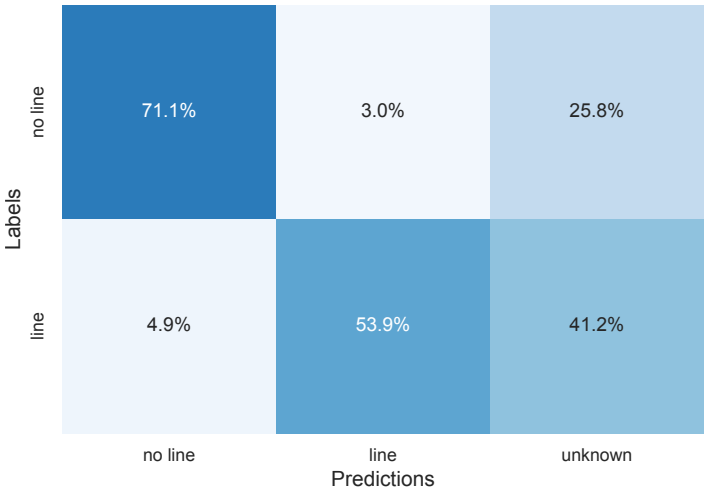
Figure S6: Example of confidence thresholds calibration for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation method. The thresholds are calibrated after training, on 4635 patches from the validation set. The confidence threshold of each class is calibrated independently. The small number of classification errors for the "*no-line*" class in the validation set makes the calibration challenging. Therefore, a value of $50\,\%$ is used as the default minimal threshold.

Model calibration methods require enough samples to cover the entire confidence spectrum, which might be challenging with a limited dataset. For example, a model with a high accuracy will correctly classify most of the samples with a high confidence score, implying that the samples with low confidence intervals will be very sparse (e.g., middle panel of Figure S6b). Therefore, it is impossible to tell whether the model is under-confident or over-confident in these sparse intervals. Fortunately, the presented uncertainty-based exploration strategy does not require the exact probability of correctness for all the confidence ranges. The purpose of the confidence score in the context of this exploration task is to optimize the exploration–exploitation tradeoff [21, 22]. When classification is performed with a confidence score under the threshold, adjacent patches will be explored. If the algorithm explores too much, it may waste time on unnecessary measurements, leading to inefficiency. Conversely, if it exploits too soon, it risks making incorrect tuning decisions based on incomplete or inaccurate information. In other words, we want to reduce the number of errors (impacting the tuning success rate) relative to the number of patches scanned (impacting the tuning time). Calibrating the threshold value by minimizing the score defined by Formula 3 is a simple way to account for the exploration–exploitation tradeoff. Fixing the meta-parameter τ to 0.2 is equivalent to saying that we prefer to have up to five more patches scanned rather than having a classification error.

Other types of models, meta-parameter values, training methods, confidence scores, and calibration methods could be evaluated to address this autotuning problem. However, an extensive benchmark of NN uncertainty is out of the scope of this paper.



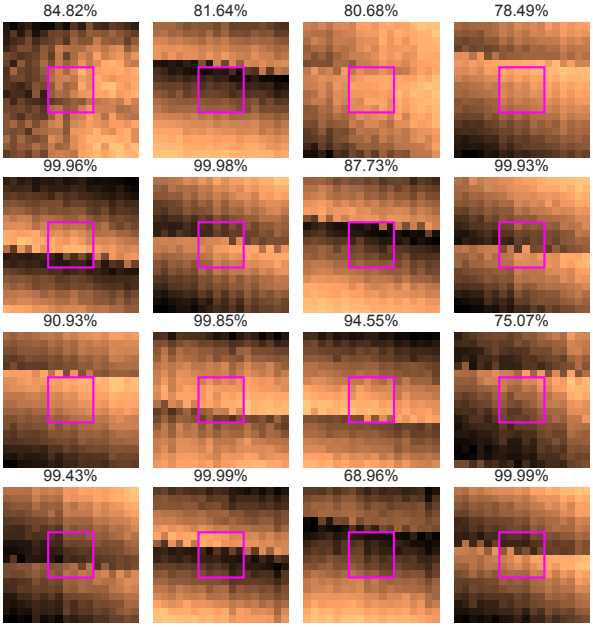
(a) Confusion matrix without using the confidence threshold. Accuracy: 81.5 %; precision: 81.0 %; recall: 84.1 %; F1 score: 80.9 %.



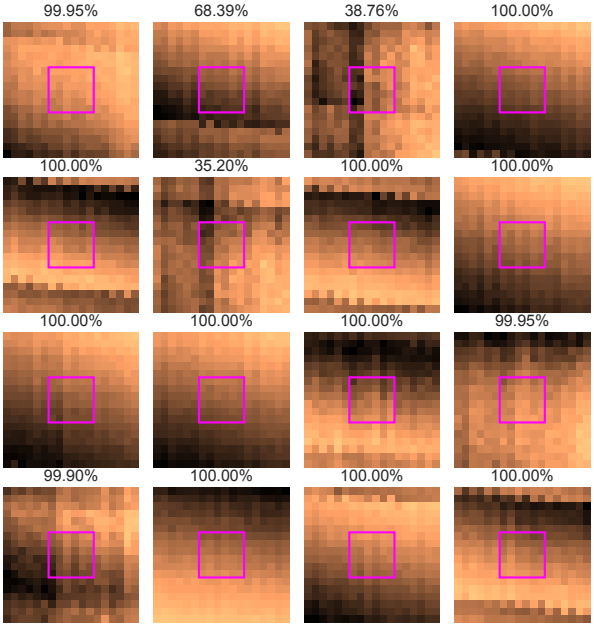
(b) Confusion matrix where every classification with a confidence score below the threshold is considered as "*unknown*". Accuracy: 94.6 %; precision: 93.5 %; recall: 93.8 %; F1 score: 93.6 %; unknown rate: 31.2 %.

Figure S7: Example of a confusion matrix for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation training method. Using the confidence threshold greatly improves the classification performance at the price of 31.2 % of the patches being labeled as "*unknown*".

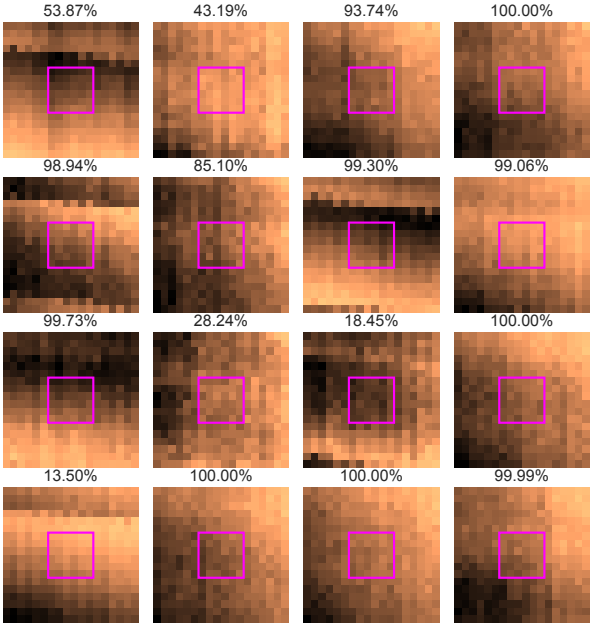
S2.3 Patch classification samples



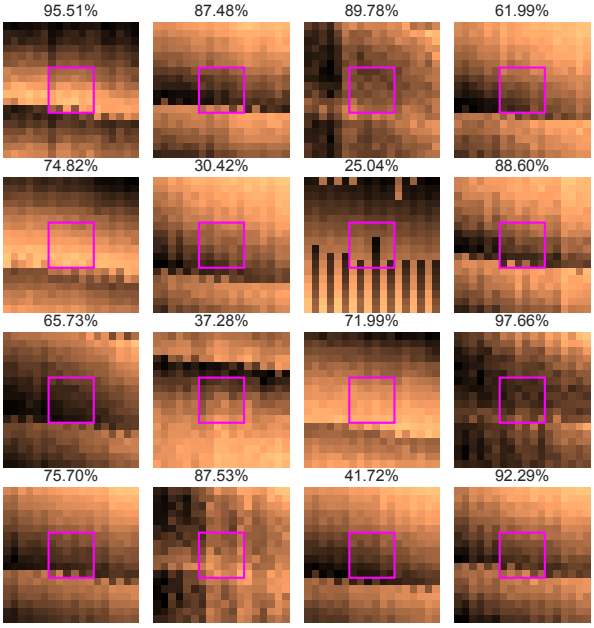
(a) Patches correctly classified as "*line*": 16 examples among the 947 patches in this case.



(b) Patches correctly classified as "*no-line*": 16 examples among the 1438 patches in this case.



(c) Patches incorrectly classified as "*no-line*" (labeled as "*line*"): 16 examples among the 75 patches in this case.



(d) Patches incorrectly classified as "*line*" (labeled as "*no-line*"): 16 examples among the 468 patches in this case.

Figure S8: Examples of patch classifications for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation training method. The patches are randomly selected among the test set. The pink squares in the center of the patches correspond to the detection area. A patch is labeled as "*line*" only if a transition line annotation crosses this square. The percentages at the top of each patch correspond to the confidence scores of the CNN computed using Formula 1.

Figure S8 presents examples of the classification results to illustrate the line detection performance. We can observe that the classification errors (Figures S8c,d) are often due to the presence of a transition line close to the detection area

or annotation mistakes. In general, the confidence scores are lower when the patch is misclassified. More samples can be found in the simulation result files available for download from Yon et al. [12].

S3 Autotuning methodology

S3.1 Exploration algorithm

The autotuning exploration algorithm is illustrated as a block diagram in Figure S9.

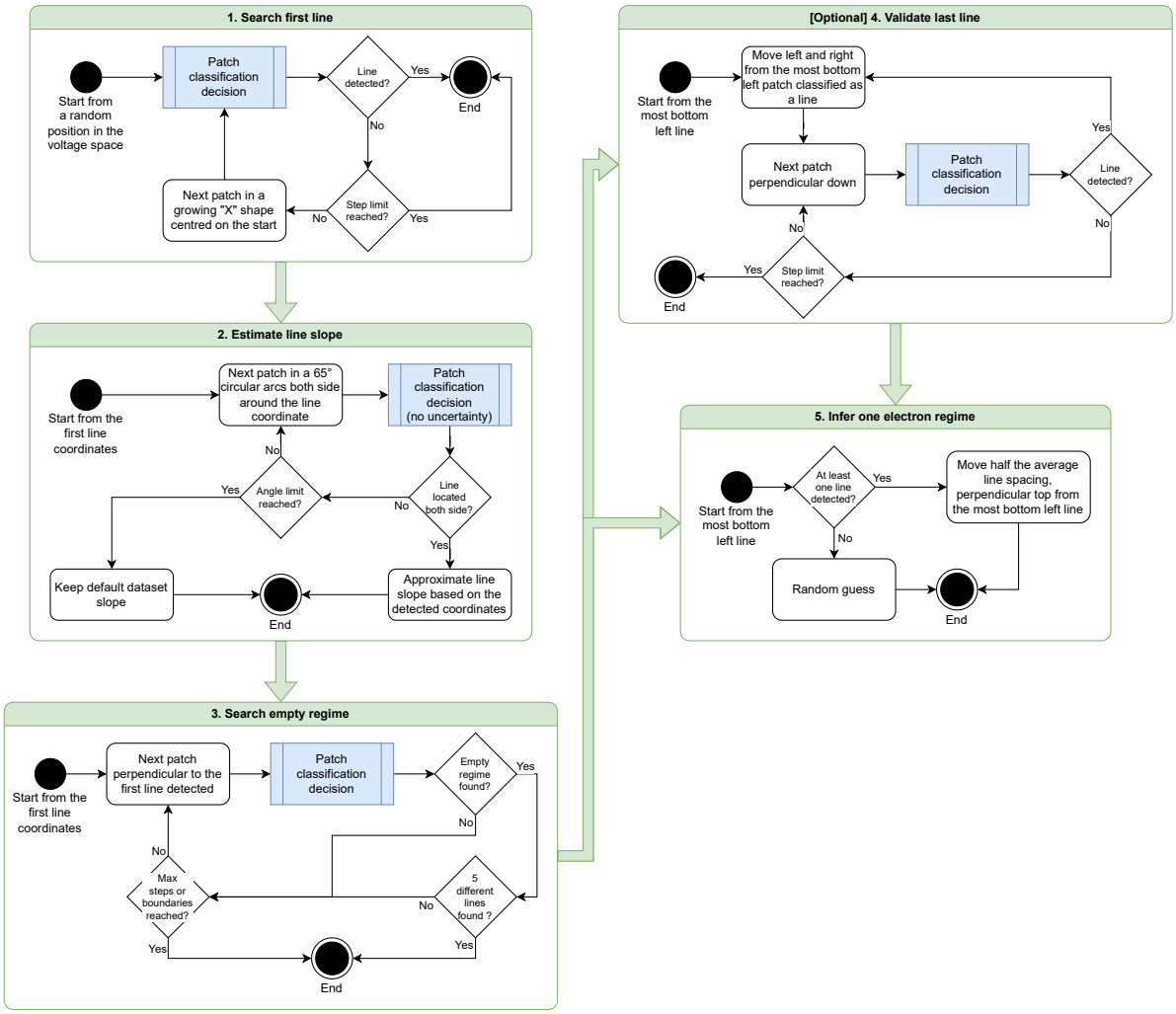


Figure S9: Detailed exploration strategy schematic representing each step of the autotuning procedure. The optional *step 4* is enabled for the GaAs-QD and Si-OG-QD datasets to improve the tuning success rate when the first transition line is fading (see examples in Figure S3).

Notes about the estimation of the transition line slope (step 2)

After finding the first patch containing a transition line, we scanned and classified a sequence of patches in circular arcs (up to $65°$) on both sides surrounding this location. This scan sequence is illustrated in step 2 of Figure 5 (the white arrows emphasize the arc directions).

We consider an arc to be valid if we detect a sequence of patches matching the following pattern:

$$m \times \textit{no-line}, n \times \textit{line}, p \times \textit{no-line}$$

where m, n, and p are positive integers representing the number of consecutive patches classified in the given label.

If both arcs are valid, we can estimate the coordinates of two points in the line by taking the position between the first and last patch coordinates in the n-th sequence of each arc. Using these two points and the *2-argument arctangent* function, we can extract the estimated slope value.

If at least one of the arcs is invalid (e.g., the line fades or a patch is misclassified), we keep the default slope value until the end of the run.

S3.2 Baselines

Two simple baselines are used as references to interpret and analyze the results of the autotuning simulations. The benchmark methodology is identical to the other autotuning procedures, including the repetition over 10 random seeds and 50 random starting points for each stability diagram.

The "*oracle*" baseline uses the exact same exploration logic as the one presented in this article, but the line detection model is replaced by an "*oracle*" that has access to the ground truth labels. Therefore, every patch classification is $100\,\%$ accurate, with a maximal confidence score. This baseline allows us to isolate the tuning errors related to the exploration strategy from the ones caused by model misclassification. The results of this baseline corroborate the conclusion that the gallium arsenide (GaAs-QD) and silicon overlapping gate (Si-OG-QD) datasets are more challenging to explore, possibly due to the transition line irregularities and unpredictable shapes.

The "*random*" baseline represents the worst case of exploration, where a pair of coordinates is randomly selected inside the voltage space of the tuned stability diagram. The success rate of this baseline can be interpreted as the average surface area of the one-electron regime in the dataset's diagrams. A larger surface is expected to be easier to find using an autotuning procedure.

Therefore, these two baselines defined the minimal and maximal tuning success rate and average number of steps expected for each dataset.

Table S3: Autotuning results using "*oracle*" and "*random*' baselines. Variability over 10 random seeds.

Dataset	Baseline	Average step number	Tuning success
Si-SG-QD	Oracle	151	100.0% ± 0.1
	Random	0	11.8% ± 1.5
GaAs-QD	Oracle	94	96.7% ± 0.6
	Random	0	10.1% ± 1.4
Si-OG-QD	Oracle	164	92.0% ± 0.9
	Random	0	5.3% ± 1.0

S4 Results without cross-validation

The performance loss observed when using the cross-validation method (illustrated in Figure S4) gives us insight into the effects of the distributional uncertainty on the autotuning success rate. The results of the line detection model trained without cross-validation, presented in Table S4, show high accuracy for every dataset (above $98\,\%$ when using the uncertainty threshold) and a consistently lower rate of instances below the threshold (number of patches classified as "*unknown*") compared to the model trained with cross-validation (presented in Table 1). As shown in Table S5, the autotuning success rates directly benefit from the model performance gain. This gap is even greater when the dataset contains very different diagrams, such as in the GaAs-QD dataset. Therefore, reducing the distributional uncertainty seems to be a promising way to improve the autotuning success rate, which could be achieved either by covering a boarder range of diagrams in the training set or by reducing the device fabrication variability.

Table S4: Line detection results without cross-validation for each dataset and model. The performances are averaged over 10 runs with different random seeds. The standard deviation of the run performances represents the variability of the methods. Each run covers every diagram of the dataset. The best test accuracy scores of each dataset are highlighted in bold.

Dataset	Model	Accuracy	Accuracy above threshold	Error reduction using threshold	Rate below threshold
Si-SG-QD	BCNN	98.8% ±0.1	**99.8%** ±0.1	79.9% ±5.1	2.0% ±0.3
	CNN	98.7% ±0.1	**99.8%** ±0.1	86.4% ±3.6	2.0% ±0.2
	FF	96.5% ±0.7	**99.8%** ±0.1	92.9% ±3.1	10.7% ±2.3
GaAs-QD	BCNN	94.3% ±0.9	96.9% ±1.1	46.5% ±15.9	8.0% ±3.1
	CNN	95.2% ±0.7	**98.0%** ±0.9	59.7% ±14.3	7.5% ±2.6
	FF	93.9% ±1.1	97.6% ±0.8	60.9% ±10.0	10.5% ±2.4
Si-OG-QD	BCNN	95.3% ±0.3	98.4% ±0.2	65.8% ±5.8	7.4% ±1.4
	CNN	93.6% ±0.4	**99.1%** ±0.2	85.2% ±2.3	11.4% ±0.9
	FF	94.1% ±0.3	98.9% ±0.2	81.0% ±3.0	11.1% ±0.7

Table S5: Autotuning results for each dataset and model without cross-validation, with and without using the model uncertainty information provided by the confidence score. The line detection accuracy and tuning success variability are computed over 10 runs with different random seeds. Each run covers every diagram of the dataset. The best tuning success rates of each dataset are highlighted in bold.

Dataset	Model	Line detection accuracy	Uncertainty-based tuning	Average step number	Tuning success
Si-SG-QD	BCNN	98.8% ±0.1	Yes	166	99.5% ±0.3
			No	151	92.2% ±2.7
	CNN	98.7% ±0.1	Yes	166	**99.9%** ±0.2
			No	152	91.2% ±3.5
	FF	96.5% ±0.7	Yes	199	71.2% ±11.7
			No	128	30.7% ±15.0
GaAs-QD	BCNN	94.3% ±0.9	Yes	101	85.4% ±9.9
			No	93	66.2% ±6.8
	CNN	95.2% ±0.7	Yes	100	**92.2%** ±2.5
			No	95	85.8% ±5.5
	FF	93.9% ±1.1	Yes	104	78.0% ±5.6
			No	92	63.0% ±3.8
Si-OG-QD	BCNN	95.3% ±0.3	Yes	184	81.8% ±2.3
			No	160	70.2% ±3.5
	CNN	93.6% ±0.4	Yes	189	84.3% ±3.0
			No	153	61.7% ±4.3
	FF	94.1% ±0.3	Yes	194	**85.0%** ±1.1
			No	154	65.5% ±2.4

References

[1] S. Rochette, M. Rudolph, A.-M. Roy, M. J. Curry, G. A. Ten Eyck, R. P. Manginell, J. R. Wendt, T. Pluym, S. M. Carr, D. R. Ward, M. P. Lilly, M. S. Carroll, and M. Pioro-Ladrière. Quantum dots with split enhancement gate tunnel barrier control. *Applied Physics Letters*, 114(8), February 2019. ISSN 1077-3118. doi:10.1063/1.5091111. URL http://doi.org/10.1063/1.5091111.

[2] L. Gaudreau, A. Kam, G. Granger, S. A. Studenikin, P. Zawadzki, and A. S. Sachrajda. A tunable few electron triple quantum dot. *Applied Physics Letters*, 95(19), November 2009. ISSN 1077-3118. doi:10.1063/1.3258663. URL http://doi.org/10.1063/1.3258663.

[3] A. Elsayed, M. M. K. Shehata, C. Godfrin, S. Kubicek, S. Massar, Y. Canvel, J. Jussot, G. Simion, M. Mongillo, D. Wan, B. Govoreanu, I. P. Radu, R. Li, P. Van Dorpe, and K. De Greve. Low charge noise quantum dots with industrial cmos manufacturing. *npj Quantum Information*, 10(1), July 2024. ISSN 2056-6387. doi:10.1038/s41534-024-00864-3. URL http://doi.org/10.1038/s41534-024-00864-3.

[4] Stefanie Czischek, Victor Yon, Marc-Antoine Genest, Marc-Antoine Roux, Sophie Rochette, Julien Camirand Lemyre, Mathieu Moras, Michel Pioro-Ladrière, Dominique Drouin, Yann Beilliard, and Roger G Melko. Miniaturizing neural networks for charge state autotuning in quantum dots. *Machine Learning: Science and Technology*, 3(1):015001, November 2021. ISSN 2632-2153. doi:10.1088/2632-2153/ac34db. URL http://doi.org/10.1088/2632-2153/ac34db.

[5] Labelbox, Inc. Labelbox, 2024. URL https://labelbox.com.

[6] Victor Yon, Bastien Galaup, Marc-Antoine Roux, Marc-Antoine Genest, Claude Rohrbacher, Joffrey Rivard, Clément Godfrin, Roy Li, Kristiaan De Greve, Sophie Rochette, Julien Camirand-Lemire, Louis Gaudreau, Michel Pioro-Ladrière, Yann Beilliard, Roger G. Melko, Eva Dupont-Ferrier, and Dominique Drouin. Quantum dots stability diagrams dataset, May 2024. URL https://doi.org/10.5281/zenodo.11402792.

[7] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks. *ArXiv*, 2015. doi:10.48550/arxiv.1505.05424. URL https://arxiv.org/abs/1505.05424. Preprint.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *ArXiv*, 2014. doi:10.48550/arxiv.1412.6980. URL https://arxiv.org/abs/1412.6980. Preprint.

[9] Sebastian Raschka. Model evaluation, model selection, and algorithm selection in machine learning. *ArXiv*, 2018. doi:10.48550/arxiv.1811.12808. URL https://arxiv.org/abs/1811.12808. Preprint.

[10] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. *ArXiv*, 2019. doi:10.48550/arxiv.1912.01703. URL https://arxiv.org/abs/1912.01703. Preprint.

[11] Piero Esposito. Blitz - bayesian layers in torch zoo (a bayesian deep learing library for torch). *GitHub repository*, 2020. URL https://github.com/piEsposito/blitz-bayesian-deep-learning.

[12] Victor Yon, Bastien Galaup, Roger G. Melko, Yann Beilliard, and Dominique Drouin. Robust Quantum Dots Charge Autotuning using Neural Network Uncertainty - Output Data, May 2024. URL https://doi.org/10.5281/zenodo.11403192.

[13] Vishal Thanvantri Vasudevan, Abhinav Sethy, and Alireza Roshan Ghias. Towards better confidence estimation for neural models. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, May 2019. doi:10.1109/icassp.2019.8683359. URL http://doi.org/10.1109/ICASSP.2019.8683359.

[14] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *ArXiv*, 2016. doi:10.48550/arxiv.1610.02136. URL https://arxiv.org/abs/1610.02136. Preprint.

[15] Murat Sensoy, Lance Kaplan, and Melih Kandemir. Evidential deep learning to quantify classification uncertainty. *ArXiv*, 2018. doi:10.48550/arxiv.1806.01768. URL https://arxiv.org/abs/1806.01768. Preprint.

[16] Marcin Możejko, Mateusz Susik, and Rafał Karczewski. Inhibited softmax for uncertainty estimation in neural networks. *ArXiv*, 2018. doi:10.48550/arxiv.1810.01861. URL https://arxiv.org/abs/1810.01861. Preprint.

[17] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the sixth annual conference on Computational learning theory*, COLT '93. ACM Press, 1993. doi:10.1145/168304.168306. URL http://doi.org/10.1145/168304.168306.

[18] Alex Graves. Practical variational inference for neural networks. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011. URL https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html.

[19] Yongchan Kwon, Joong-Ho Won, Beom Joon Kim, and Myunghee Cho Paik. Uncertainty quantification using bayesian neural networks in classification: Application to biomedical image segmentation. *Computational Statistics & Data Analysis*, 142:106816, February 2020. ISSN 0167-9473. doi:10.1016/j.csda.2019.106816. URL http://doi.org/10.1016/j.csda.2019.106816.

[20] Laurent Valentin Jospin, Hamid Laga, Farid Boussaid, Wray Buntine, and Mohammed Bennamoun. Hands-on bayesian neural networks—a tutorial for deep learning users. *IEEE Computational Intelligence Magazine*, 17(2): 29–48, May 2022. ISSN 1556-6048. doi:10.1109/mci.2022.3155327. URL http://doi.org/10.1109/MCI.2022.3155327.

[21] Peter Auer. Using confidence bounds for exploitation-exploration trade-offs. *Journal of Machine Learning Research*, 3(Nov):397–422, 2002. URL https://www.jmlr.org/papers/v3/auer02a.html.

[22] Alex Simpkins, Raymond de Callafon, and Emanuel Todorov. Optimal trade-off between exploration and exploitation. In *2008 American Control Conference*. IEEE, jun 2008. doi:10.1109/ACC.2008.4586462. URL https://doi.org/10.1109/ACC.2008.4586462e.